

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, Input
```

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype("float32")/255.0
x_test = x_test.astype("float32")/255.0
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
 11490434/11490434 ————— 0s 0us/step

```
x_train.shape,x_test.shape
```

```
((60000, 28, 28), (10000, 28, 28))
```

```
x_train = np.expand_dims(x_train,-1)
x_test = np.expand_dims(x_test,-1)
```

```
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0,scale=1.0,size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0,scale=1.0,size=x_test.shape)
```

```
x_train_noisy = np.clip(x_train_noisy,0.,1.)
x_test_noisy = np.clip(x_test_noisy,0.,1.)
```

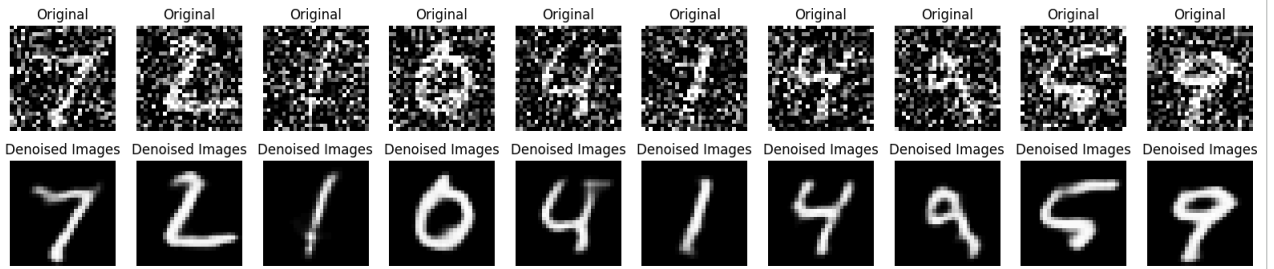
```
autoencoder = Sequential()
autoencoder.add(Conv2D(32,(3,3),activation="relu",padding="same"))
autoencoder.add(MaxPooling2D((2,2),padding="same"))
autoencoder.add(Conv2D(64,(3,3),activation="relu",padding="same"))
autoencoder.add(MaxPooling2D((2,2),padding="same"))
autoencoder.add(Conv2D(64,(3,3),activation="relu",padding="same"))
autoencoder.add(UpSampling2D((2,2)))
autoencoder.add(Conv2D(32,(3,3),activation="relu",padding="same"))
autoencoder.add(UpSampling2D((2,2)))
autoencoder.add(Conv2D(1,(3,3),activation="sigmoid",padding="same"))
autoencoder.compile(optimizer="adam",loss="binary_crossentropy",metrics=["accuracy"])
history = autoencoder.fit(x_train_noisy,x_train,epochs=10,batch_size=256,validation_data=(x_test_noisy,x_test))
```

```
Epoch 1/10
235/235 ————— 10s 25ms/step - accuracy: 0.8059 - loss: 0.2718 - val_accuracy: 0.8083 - val_loss: 0.1194
Epoch 2/10
235/235 ————— 3s 12ms/step - accuracy: 0.8102 - loss: 0.1174 - val_accuracy: 0.8113 - val_loss: 0.1091
Epoch 3/10
235/235 ————— 3s 12ms/step - accuracy: 0.8118 - loss: 0.1094 - val_accuracy: 0.8116 - val_loss: 0.1051
Epoch 4/10
235/235 ————— 3s 12ms/step - accuracy: 0.8124 - loss: 0.1055 - val_accuracy: 0.8122 - val_loss: 0.1031
Epoch 5/10
235/235 ————— 3s 12ms/step - accuracy: 0.8123 - loss: 0.1034 - val_accuracy: 0.8120 - val_loss: 0.1011
Epoch 6/10
235/235 ————— 3s 12ms/step - accuracy: 0.8126 - loss: 0.1019 - val_accuracy: 0.8125 - val_loss: 0.1008
Epoch 7/10
235/235 ————— 3s 12ms/step - accuracy: 0.8131 - loss: 0.1007 - val_accuracy: 0.8122 - val_loss: 0.0992
Epoch 8/10
235/235 ————— 4s 15ms/step - accuracy: 0.8130 - loss: 0.0999 - val_accuracy: 0.8121 - val_loss: 0.0985
Epoch 9/10
235/235 ————— 3s 12ms/step - accuracy: 0.8132 - loss: 0.0991 - val_accuracy: 0.8121 - val_loss: 0.0980
Epoch 10/10
235/235 ————— 3s 13ms/step - accuracy: 0.8132 - loss: 0.0986 - val_accuracy: 0.8122 - val_loss: 0.0975
```

```
denoised_imgs = autoencoder.predict(x_test_noisy)
n = 10
plt.figure(figsize=(20,4))
for i in range(n):
    ax = plt.subplot(2,n,i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28),cmap="gray")
    plt.title("Original")
    plt.axis("off")

    ax = plt.subplot(2,n,i+1+n)
    plt.imshow(denoised_imgs[i].reshape(28,28),cmap="gray")
    plt.title("Denoised Images")
    plt.axis("off")
plt.show()
```

313/313 — 2s 3ms/step

Start coding or [generate](#) with AI.